

Note: All problems must be implemented in C/C++ unless stated otherwise.

Problem 1 — Median via Selection Sort Invariant

Selection sort works by repeatedly finding the minimum element from the unsorted portion and placing it at the correct position. After k passes, the k smallest elements occupy indices 0 through $k - 1$ in sorted order.

Use this invariant to find the median of an unsorted array without sorting the whole array. The median is the middle element when the array has an odd number of elements, or the lower-middle element when the count is even (i.e. element at index $\text{size}/2$ after partial sorting).

Function Signature:

```
int find_median(int *arr, int size);
```

Constraints: $1 \leq \text{size} \leq 10^5$, $-10^9 \leq \text{arr}[i] \leq 10^9$

Example 1:

Input : arr = {7, 2, 10, 9, 1}, size = 5

Output: 7

Explanation: Sorted = {1, 2, 7, 9, 10}. Middle index = 2. Median = 7.

Example 2:

Input : arr = {4, 1, 8, 3}, size = 4

Output: 3

Explanation: Sorted = {1, 3, 4, 8}. Lower-middle index = 1. Median = 3.

Note: You only need to run selection sort until you reach index $\text{size}/2$. Stop there.

Problem 2 — K-th Largest via Bubble Sort Invariant

Bubble sort works by repeatedly bubbling the largest unsorted element to the end of the array. After k passes, the k largest elements are correctly placed at the tail of the array.

Use this invariant to find the k -th largest element by running only as many passes as needed.

Function Signature:

```
int find_kth_largest(int *arr, int size, int k);
```

Constraints: $1 \leq k \leq \text{size} \leq 10^5$

Example 1:

Input : arr = {3, 1, 7, 4, 9, 2}, size = 6, k = 2

Output: 7

Explanation: Sorted descending = {9, 7, 4, 3, 2, 1}. 2nd largest = 7.

Example 2:

Input : arr = {10, 5, 8, 3}, size = 4, k = 4

Output: 3

Explanation: 4th largest is the smallest element = 3.

Note: Run exactly k bubble passes. The k -th largest will be at index $\text{size} - k$ after k passes.

Problem 3 — Merge Two Arrays via Insertion Sort Logic

Insertion sort builds a sorted sequence by taking one element at a time and inserting it into its correct position among previously processed elements.

Given two unsorted arrays A and B , treat all elements from both as a combined stream of keys. Insert each key one by one into an output array, maintaining ascending sorted order at all times. The output array should contain all $n + m$ elements in sorted order.

Function Signature:

```
void merge_sorted(int *a, int n, int *b, int m, int *out);
```

Example:

Input : $a = \{5, 1, 9\}$, $n = 3$

$b = \{3, 7, 2, 6\}$, $m = 4$

Output: $out = \{1, 2, 3, 5, 6, 7, 9\}$

Note: You may process elements from A and B in any order. The key constraint is that each new element must be inserted into the correct position in $out[]$, shifting elements as needed.

Problem 4.1 — Factorial

Compute $n! = n \times (n - 1) \times \cdots \times 1$. Define $0! = 1$.

Function Signature:

```
long long factorial(int n);
```

Examples:

```
factorial(0) => 1
```

```
factorial(5) => 120
```

```
factorial(10) => 3628800
```

Note: Assume $n \geq 0$. For large n , the result may overflow a 32-bit integer — use `long long`.

Problem 4.2 — Fibonacci

Compute the n -th Fibonacci number where $\text{fib}(0) = 0$, $\text{fib}(1) = 1$, and $\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$.

Function Signature:

```
long long fibonacci(int n);
```

Examples:

```
fibonacci(0) => 0
```

```
fibonacci(7) => 13
```

```
fibonacci(10) => 55
```

Note: This naive recursive approach has $O(2^n)$ time complexity. Observe how slow it gets for $n > 40$.

Problem 4.3 — Count Digits

Count the number of digits in a non-negative integer. The number 0 has exactly 1 digit.

Function Signature:

```
int count_digits(int n);
```

Examples:

```
count_digits(0)    => 1
count_digits(9)    => 1
count_digits(100)  => 3
count_digits(98765) => 5
```

Note: Hint: Repeatedly divide by 10. The recursive step reduces n by one digit each call.

Problem 4.4 — Power Function

Compute `base` raised to the power `exp` (i.e. `baseexp`). Assume `exp` ≥ 0 and define `base0` = 1.

Function Signature:

```
double power(double base, int exp);
```

Examples:

```
power(2.0, 0) => 1.0
power(3.0, 4) => 81.0
power(2.5, 3) => 15.625
```

Note: A simple $O(\text{exp})$ solution is acceptable. For a challenge, implement it in $O(\log \text{exp})$ using fast exponentiation.

Problem 4.5 — Palindrome Check

Return 1 if the given string is a palindrome (reads the same forwards and backwards), 0 otherwise. Comparison is case-sensitive.

Function Signature:

```
int is_palindrome(char *str, int left, int right);
```

Examples:

```
is_palindrome("racecar", 0, 6) => 1
is_palindrome("hello", 0, 4) => 0
is_palindrome("a", 0, 0) => 1
is_palindrome("abba", 0, 3) => 1
```

Note: Use two indices (`left`, `right`) that move inward. The base case is when `left` $\geq$ `right`.

Problem 4.6 — Sum of Array

Return the sum of all elements in an integer array of given size.

Function Signature:

```
int array_sum(int *arr, int size);
```

Examples:

```
array_sum({1,2,3,4,5}, 5) => 15
array_sum({-3, 7, 0}, 3)  => 4
array_sum({42}, 1)       => 42
```

Note: Reduce the problem by summing the last element with the result of the remaining subarray.

Problem 4.7 — Maximum of Array

Return the maximum element in an integer array of given size.

Function Signature:

```
int array_max(int *arr, int size);
```

Examples:

```
array_max({3, 1, 9, 2, 7}, 5) => 9
array_max({-5, -1, -8}, 3)    => -1
array_max({42}, 1)           => 42
```

Note: Base case: an array of size 1 has its only element as the maximum.

Problem 4.8 — Binary Search

Search for a target value in a sorted array. Return the index of the target if found, or -1 if not present. The input array is guaranteed to be sorted in ascending order.

Function Signature:

```
int binary_search(int *arr, int left, int right, int target);
```

Examples:

```
binary_search({1,3,5,7,9}, 0, 4, 7) => 3
binary_search({1,3,5,7,9}, 0, 4, 4) => -1
binary_search({2}, 0, 0, 2)        => 0
```

Note: Compute $\text{mid} = (\text{left} + \text{right}) / 2$, then recurse into the appropriate half.

Problem 5 — Merge Sort

Implement merge sort, a divide-and-conquer algorithm with guaranteed $O(n \log n)$ worst-case time complexity. The algorithm splits the array into two halves, recursively sorts each half, then merges them back together. Your implementation must sort the array in-place (or use a temporary buffer only during the merge step).

Function Signatures:

```
void merge_sort(int *arr, int left, int right);  
void merge(int *arr, int left, int mid, int right);
```

Example:

```
Input : {38, 27, 43, 3, 9, 82, 10}  
Output: {3, 9, 10, 27, 38, 43, 82}  
Trace (first two levels of recursion):  
Split: {38,27,43,3} | {9,82,10}  
Split: {38,27} | {43,3} and {9,82} | {10}  
Merge pairs, then merge halves, then merge full array.
```

Requirements:

1. The merge step must run in $O(n)$ time using an auxiliary array.
2. Your merge_sort must be called with merge_sort(arr, 0, size-1).
3. Verify correctness on at least: an already-sorted array, a reverse-sorted array, and an array with duplicates.

Note: Time complexity: $O(n \log n)$ in all cases. Space complexity: $O(n)$ auxiliary.

Problem 6 — Quick Sort

Implement quick sort using the Lomuto partition scheme. Choose the last element as the pivot. Partition the array so that all elements smaller than the pivot come before it and all larger elements come after. Then recursively sort the two sub-partitions.

Function Signatures:

```
void quick_sort(int *arr, int low, int high);  
int partition(int *arr, int low, int high);
```

Example:

```
Input : {10, 7, 8, 9, 1, 5}  
Pivot (last) = 5  
After partition: {1, 5, 8, 9, 7, 10} (5 is now at correct index 1)  
Recurse on {1} and {8, 9, 7, 10}  
Final Output : {1, 5, 7, 8, 9, 10}
```

Requirements:

1. partition() must return the final index of the pivot.
2. Test on a reverse-sorted array (worst case for naive pivot choice).
3. Count and print the total number of swaps performed.

Note: Average-case time: $O(n \log n)$. Worst-case: $O(n^2)$ with last-element pivot on sorted input.

Problem 7 — Fractional Knapsack

You are given n items, each with a weight and a value, and a knapsack with a maximum weight capacity W . Unlike the 0/1 knapsack problem, you may take fractions of an item. Your goal is to maximize the total value of items in the knapsack.

Greedy strategy: sort items by value-to-weight ratio in descending order. Take as much of the highest-ratio item as possible, then move to the next.

Struct Definition:

```
typedef struct {
    int weight;
    int value;
} Item;
```

Function Signature:

```
double fractional_knapsack(Item *items, int n, int capacity);
```

Example 1:

```
Items: weight = {10, 20, 30}, value = {60, 100, 120}
Capacity: W = 50
Ratios: 6.0, 5.0, 4.0 => Sort: Item0, Item1, Item2
Take all of Item0 (10 kg, +60), all of Item1 (20 kg, +100),
and 20/30 of Item2 (+80).
Output: 240.00
```

Example 2:

```
Items: weight = {5, 10, 15}, value = {50, 60, 90}
Capacity: W = 20
Output: 160.00
```

Note: Print the result to 2 decimal places. You may not carry more than capacity kg in total.

Problem 8 — Minimum Spanning Tree

A Minimum Spanning Tree (MST) of a weighted undirected graph is a spanning tree whose total edge weight is minimized. Implement both Prim's and Kruskal's algorithms and verify they produce MSTs of the same total weight on the same input.

Represent the graph as an adjacency matrix for Prim's and as an edge list for Kruskal's.

8a — Prim's Algorithm: Start from vertex 0. Greedily add the minimum-weight edge that connects a visited vertex to an unvisited one. Repeat until all vertices are included.

```
void prims_mst(int graph[V][V], int V);
```

8b — Kruskal's Algorithm: Sort all edges by weight. Use a Union-Find (Disjoint Set Union) structure to add edges greedily without forming cycles.

```
void kruskals_mst(Edge *edges, int E, int V);
```

Example Graph (5 vertices):

Edge list: (0-1, w=2), (0-3, w=6), (1-2, w=3), (1-3, w=8),
(1-4, w=5), (2-4, w=7), (3-4, w=9)

MST edges: (0-1, 2), (1-2, 3), (1-4, 5), (0-3, 6)

Total MST weight: 16

Required Output Format:

Edge Weight

0 - 1 2

1 - 2 3

...

Total weight: 16

Note: Kruskal's requires a Union-Find with union-by-rank or path compression for full credit.

Problem 9a — Dijkstra's Algorithm

Dijkstra's algorithm finds shortest paths in graphs with non-negative edge weights. Use a greedy approach: always relax edges from the unvisited vertex with the smallest known distance.

```
void dijkstra(int graph[V][V], int src, int V);
```

Example Graph (5 vertices, adjacency matrix, 0 = no edge):

```
      0  1  2  3  4
0 [ 0, 4, 0, 0, 8 ]
1 [ 4, 0, 8, 0, 11]
2 [ 0, 8, 0, 7, 0 ]
3 [ 0, 0, 7, 0, 9 ]
4 [ 8,11, 0, 9, 0 ]
```

Output (source = 0):

Vertex Distance from Source

```
0      0
1      4
2     12
3     19
4      8
```

Note: This algorithm does not work correctly with negative edge weights. For that, see Bellman-Ford.

Problem 9b — Bellman-Ford Algorithm

Bellman-Ford computes shortest paths even when some edge weights are negative. It also detects negative-weight cycles and reports if one is reachable from the source. Relax all edges $V - 1$ times. On the V -th relaxation, if any distance still decreases, a negative cycle exists.

```
void bellman_ford(int V, int E, Edge *edges, int src);
```

Example Graph (5 vertices):

Edges: (0->1, w=6), (0->2, w=7), (1->2, w=8), (1->3, w=5),
 (1->4, w=-4), (2->3, w=-3), (2->4, w=9), (3->1, w=-2),
 (4->0, w=2), (4->3, w=7)

Output (source = 0):

Vertex	Distance from Source
0	0
1	2
2	7
3	4
4	-2

Negative Cycle Detection:

If a negative cycle exists, print: "Graph contains a negative-weight cycle"

Note: Represent INF (unreachable) as a large constant, e.g. `INT_MAX / 2`, to avoid overflow during relaxation.

Problem 10a — 0/1 Knapsack

You are given n items each with an integer weight and value, and a knapsack of capacity W . Each item can either be taken (1) or left (0) — you cannot take fractions. Maximize the total value without exceeding the weight capacity.

Recurrence:

```
dp[i][w] = max value using first i items with capacity w
          = max(dp[i-1][w], dp[i-1][w - weight[i]] + value[i])
              if weight[i] <= w, else dp[i-1][w]
```

Function Signature:

```
int knapsack_01(int *weights, int *values, int n, int W);
```

Example 1:

weights = {2, 3, 4, 5}, values = {3, 4, 5, 6}, n = 4, W = 5
 Best choice: item 0 (w=2, v=3) + item 1 (w=3, v=4)
 total weight 5, value 7
 Output: 7

Example 2:

weights = {1, 3, 4, 5}, values = {1, 4, 5, 7}, n = 4, W = 7
 Output: 9

Problem 10b — Minimum Number of Coins

Given an array of coin denominations and a target amount, find the minimum number of coins needed to make up that amount. You have an unlimited supply of each denomination. Return -1 if the amount cannot be formed.

Recurrence:

```
dp[0] = 0
dp[amount] = min over all coins c where c <= amount:
    1 + dp[amount - c]
```

Function Signature:

```
int min_coins(int *coins, int n, int amount);
```

Examples:

```
coins = {1, 5, 6, 9}, amount = 11 => Output: 3
coins = {2}, amount = 3           => Output: -1 (impossible)
coins = {1, 2, 5}, amount = 11   => Output: 3 (5 + 5 + 1)
```

Problem 10c — Minimum Ways of Coin Change (Count of Combinations)

Given coin denominations and a target amount, count the total number of distinct combinations of coins that sum to the target. Order does not matter — {1,2} and {2,1} count as one combination. You have an unlimited supply of each denomination.

Recurrence:

```
dp[0] = 1 (one way to make 0: use no coins)
For each coin c, for each amount a from c to amount:
    dp[a] += dp[a - c]
```

Function Signature:

```
int count_ways(int *coins, int n, int amount);
```

Examples:

```
coins = {1, 2, 3}, amount = 4
Combinations: {1,1,1,1}, {1,1,2}, {1,3}, {2,2} => Output: 4

coins = {2, 5, 3, 6}, amount = 10 => Output: 5
```

Note: The outer loop must iterate over coins (not amounts) to avoid counting permutations as distinct.

Problem 10d — Longest Common Subsequence (LCS)

Given two strings `s1` and `s2`, find the length of their longest common subsequence. A subsequence is formed by deleting some (or no) characters from a string without changing the relative order of remaining characters. Characters do not need to be contiguous.

Recurrence:

```
dp[i][j] = LCS length of s1[0..i-1] and s2[0..j-1]
          = dp[i-1][j-1] + 1 if s1[i-1] == s2[j-1]
          = max(dp[i-1][j], dp[i][j-1]) otherwise
```

Function Signature:

```
int lcs(char *s1, char *s2, int m, int n);
```

Examples:

```
s1 = "ABCBDAB", s2 = "BDCAB"    => Output: 4
s1 = "AGGTAB",  s2 = "GTXAYB"   => Output: 4
s1 = "ABC",     s2 = "AC"        => Output: 2
```

Note: For bonus credit, reconstruct and print the actual LCS string by backtracking through the `dp` table.

Problem 10e — Longest Increasing Subsequence (LIS)

Given an array of integers, find the length of the longest strictly increasing subsequence. A subsequence is increasing if each subsequent element is strictly greater than the previous one. Elements do not need to be contiguous in the original array.

Approach ($O(n^2)$ DP):

```
dp[i] = length of LIS ending at index i
dp[i] = 1 + max(dp[j]) for all j < i where arr[j] < arr[i]
Answer = max(dp[i]) over all i
```

Function Signature:

```
int lis(int *arr, int n);
```

Examples:

```
Input : {10, 9, 2, 5, 3, 7, 101, 18}
Output: 4
```

```
Input : {0, 1, 0, 3, 2, 3}
Output: 4
```

```
Input : {7, 7, 7, 7}
Output: 1
```

Note: For bonus credit, implement the $O(n \log n)$ patience sorting approach using binary search.